

Domain-Constrained Retrieval: Failure Prevention and Curation Signals for Typed Knowledge Bases

Raj Navakoti
rajnavakoti@gmail.com

June 2026

Abstract

Retrieval-augmented generation (RAG) systems typically treat knowledge bases as flat document collections, ignoring structural information about entity types, knowledge layers, and typed relationships. When deployed against typed, layered knowledge bases—where entities are categorized by type and organized into semantic layers—this structural blindness leads to silent retrieval failures: the system returns plausible but incomplete or irrelevant context, and neither the user nor the system knows what was missed. We present **Domain-Constrained Retrieval (DCR)**, a seven-stage retrieval pipeline designed for typed knowledge bases. DCR introduces intent-aware query understanding, typed graph traversal, confidence-weighted scoring, multi-layer deduplication, and—as its primary novel contribution—**gap detection**, which converts retrieval dead-ends into actionable curation signals. We evaluate DCR against two baselines (naive vector search and vanilla RAG) on a synthetic healthcare claims knowledge base containing 408 entities across 18 types and 6 layers, using 50 evaluation questions across 3 independent trials per condition. DCR significantly reduces catastrophic retrieval failures: MISSING answers drop from 12.7% (vanilla RAG) to 4.7% (χ^2 test, $p = 0.024$). On single-layer questions where the knowledge exists, DCR achieves 1.0% MISSING compared to 12.5% for vanilla RAG. Stage-by-stage ablation reveals that retrieval stage contributions are density-dependent: intent classification and graph traversal add noise on dense knowledge bases (408 entities) but are hypothesized to help on bounded-context knowledge bases (50–80 entities). Gap detection fires on 82% of questions regardless of answer quality, acting as a continuous knowledge base health probe rather than a retrieval confidence signal; preliminary evidence suggests high-severity coverage gaps may predict retrieval failure, though this requires validation at scale. We report per-question stability across trials and find that pipeline complexity inversely correlates with scoring consistency—a methodological observation with implications for RAG evaluation practice.

Keywords: retrieval-augmented generation, typed knowledge bases, gap detection, domain-constrained retrieval, knowledge curation, entity-relationship retrieval

1 Introduction

Retrieval-augmented generation has become the dominant approach for grounding large language models in domain-specific knowledge [Lewis et al., 2020]. The standard RAG pipeline embeds documents as vectors, retrieves the top- k most similar chunks to a query, and passes them as context to a generative model. This works well when the knowledge base is a flat collection of documents with uniform structure.

Enterprise knowledge bases are rarely flat. In domains such as healthcare claims processing, financial operations, and supply chain management, knowledge is inherently *typed*: systems differ from processes, APIs differ from data models, business terminology differs from technical

jargon. These entities exist in typed relationships—a system implements a capability, a process triggers events, a team owns a service—and are organized into semantic layers ranging from structural definitions to behavioral workflows to organizational ownership.

When a standard RAG pipeline encounters such a knowledge base, it has no mechanism to exploit this structure. Vector similarity treats a system entity and a jargon definition identically. There is no way to traverse typed relationships to discover relevant context that is semantically distant but structurally connected. Most critically, there is no way to detect what the retrieval *missed*: if a question requires knowledge about a process that coordinates two systems, and the knowledge base documents the systems but not the coordination, the pipeline returns the system entities and produces a plausible-sounding but incomplete answer. The failure is silent.

We present **Domain-Constrained Retrieval (DCR)**, a seven-stage retrieval pipeline designed for typed, layered knowledge bases. DCR addresses three problems that standard RAG ignores:

1. **Type-aware retrieval.** DCR classifies query intent to determine which entity types and knowledge layers are most relevant, uses typed graph traversal to follow relationship edges, and applies layer-priority boosting during fusion.
2. **Failure prevention.** Confidence-weighted scoring adjusts entity relevance based on content density, preventing the pipeline from returning shallow or poorly-documented entities when richer alternatives exist.
3. **Gap detection.** DCR’s novel contribution: converting retrieval dead-ends— orphan entities with no relationships, ambiguous type assignments, thin coverage areas—into structured signals that tell knowledge curators what to improve.

We evaluate DCR on a synthetic healthcare claims knowledge base containing 408 entities across 18 types and 6 knowledge layers. Our experimental design uses 50 evaluation questions, 9 retrieval conditions (3 baselines + 6 ablation), and 3 independent trials per condition (1,350 total evaluations). We report three main findings:

1. DCR significantly reduces catastrophic retrieval failures (MISSING answers: 4.7% vs. 12.7% for vanilla RAG, $p = 0.024$), though overall answer quality improvement is not statistically significant ($p = 0.059$).
2. The MISSING reduction concentrates in single-layer questions (12.5% $\rightarrow$ 1.0%), while cross-cutting questions remain at $\sim 83\%$ INCOMPLETE regardless of retrieval method—exposing knowledge base content gaps that retrieval cannot fix.
3. Stage contributions are density-dependent: intent classification and graph traversal reduce accuracy on this dense knowledge base, while confidence scoring is the load-bearing stage whose removal doubles MISSING on cross-cutting questions.

The remainder of this paper is organized as follows. Section 2 surveys related work. Section 3 describes the DCR pipeline architecture. Section 4 details the experimental setup. Sections 5 and 6 present results and analysis. Section 7 discusses implications, and Section 8 concludes.

2 Related Work

2.1 Retrieval-Augmented Generation

RAG systems augment language model generation with retrieved context to improve factual grounding [Lewis et al., 2020]. Early systems use dense retrieval with a single vector index. More recent work introduces hybrid retrieval combining dense and sparse methods [Chen et al., 2024], multi-step retrieval with query decomposition [Press et al., 2023], and self-reflective retrieval where the model decides whether to retrieve [Asai et al., 2024]. These advances improve retrieval quality on unstructured document collections but do not exploit typed entity structure.

2.2 Knowledge Graph-Enhanced Retrieval

Graph-based retrieval augments vector search with structured knowledge. GraphRAG [Edge et al., 2024] builds community summaries from document graphs for global queries. GRAG [Hu et al., 2024] integrates subgraph retrieval with text generation. These approaches construct graphs from unstructured text via entity extraction. DCR differs fundamentally: it operates on *pre-existing* typed knowledge bases where entity types, relationships, and layers are defined by a domain meta-model, not extracted from text. This enables type-aware query understanding and typed edge traversal that post-hoc extraction cannot support.

2.3 Retrieval Evaluation

LLM-as-judge evaluation [Zheng et al., 2023] is widely used but introduces scoring variance. Shankar et al. [2024] show that single-judge setups have low inter-rater reliability. Cons-Judge [Liu et al., 2025] trains judges via DPO on consistent vs. inconsistent judgment pairs to improve reliability. Our evaluation uses a single LLM judge with multi-trial averaging to characterize variance, and we report inter-rater reliability (Cohen’s $\kappa = 0.634$) as a methodological contribution.

2.4 Enterprise and Domain-Specific RAG

Enterprise RAG applications face challenges distinct from open-domain QA: structured domain knowledge, specialized terminology, regulatory constraints, and the need to audit retrieval provenance. Recent work addresses RAG for regulatory compliance [Sarmah et al., 2024], medical knowledge retrieval, and code-specific contexts. However, these approaches typically adapt general RAG to domain-specific *content* while retaining flat retrieval architectures. DCR differs by adapting to domain-specific *structure*—exploiting entity types, knowledge layers, and typed relationships that are inherent to enterprise knowledge organization.

2.5 Schema-Aware Retrieval

A parallel line of work exploits typed schemas at query time. NL2SQL systems [Rajkumar et al., 2022] use database schemas to constrain query generation; enterprise search engines use faceted metadata (document type, department, date) to filter results. DCR extends this principle to knowledge base retrieval: entity types and relationship schemas constrain intent classification, graph traversal, and gap detection. Unlike NL2SQL, DCR operates on semi-structured entity descriptions rather than relational tables, and unlike faceted search, it uses schema structure for *retrieval scoring*, not just filtering.

2.6 Gap Detection in Information Retrieval

Knowledge base completeness assessment is studied in the context of knowledge graph completion [Bordes et al., 2013] and open-domain QA [Rajpurkar et al., 2018]. However, these approaches focus on *what the model cannot answer* rather than *what the knowledge base is missing*. DCR’s gap detection operates at the retrieval layer, identifying structural deficiencies (orphan entities, ambiguous types, thin coverage) independently of answer quality. This positions gap detection as a curation signal rather than an evaluation metric.

3 The DCR Pipeline

DCR operates on typed knowledge bases structured according to a domain meta-model. The meta-model defines entity types (e.g., *system*, *process*, *team*, *API*), knowledge layers (e.g., *structural*, *behavioral*, *organizational*), and typed relationships between entities. Each entity has

Structural	<i>What exists?</i>	system software-component platform	142
Behavioral	<i>How does it work?</i>	process business-event api domain-logic	118
Reference	<i>What are the values?</i>	data-model data-product reference-data	67
Organizational	<i>Who is involved?</i>	team persona offering	41
Language	<i>What do terms mean?</i>	jargon-business jargon-tech	28
Decision	<i>Why was this chosen?</i>	decision	12

6 layers • 18 entity types • 408 entities • 5 typed relationship types

Figure 1: The typed, layered knowledge base meta-model. Each row groups entity types that answer a category of domain question. Entities carry typed relationships across layers (e.g., `depends_on`, `owned_by`). DCR exploits this structure for intent classification, graph traversal, and gap detection.

a type, a layer assignment, a confidence score reflecting curation quality, and a set of typed relationships to other entities.

Figure 1 illustrates the meta-model structure. The pipeline processes a query through seven stages. Figure 2 shows the architecture.

3.1 Stage 1: Intent Classification

Intent classification determines which entity types and knowledge layers are most relevant to the query. DCR uses a zero-LLM approach: regex pattern matching against the query text to assign weights to eight fixed, hand-designed intent categories (process, ownership, entity, relationship, temporal, diagnostic, listing, comparison). These categories are not learned; they were derived from manual analysis of 200+ enterprise support questions across three domains during prior work on the Demand-Driven Context methodology [Navakoti and Navakoti, 2026]. Each intent maps to a weight vector over entity types and knowledge layers. For example, a process query increases weights for *process*, *business-event*, and *workflow* entity types and boosts the *behavioral* knowledge layer.

3.2 Stage 2: Parallel Search

Three search strategies run concurrently:

- **Vector search.** The query is embedded and compared against entity embeddings using cosine similarity. Returns the top- K candidates, where K is automatically scaled to 20% of entity count (81 entities for this knowledge base). This threshold balances recall (surfacing relevant entities across types) against context window limits; we acknowledge this default was not ablated and note it as a limitation.
- **Keyword search.** BM25-style scoring against entity names and body text.
- **Graph search.** Breadth-first traversal from the top vector results, following typed relationship edges. Edge types are filtered based on query intent: a process query follows `triggers`, `depends_on`, and `implements` edges but not `owned_by`. Maximum depth is 3 hops.

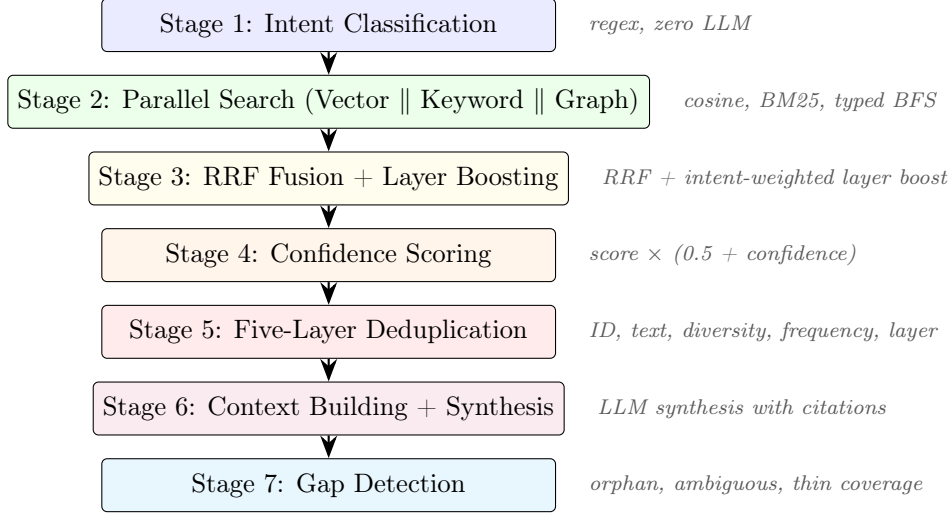


Figure 2: The seven-stage DCR pipeline. Stages 1–5 handle retrieval; Stage 6 synthesizes an answer; Stage 7 detects knowledge base gaps. All stages except Stage 6 are deterministic.

3.3 Stage 3: RRF Fusion with Layer Boosting

Reciprocal Rank Fusion [Cormack et al., 2009] merges the three candidate lists. The fused score is blended with the original cosine similarity: $s = 0.7 \times s_{\text{RRF}} + 0.3 \times s_{\text{cosine}}$. Layer priority boosting then adjusts scores based on the intent-derived layer weights from Stage 1.

3.4 Stage 4: Confidence Scoring

Each entity carries a confidence score (0.0–1.0) reflecting its curation quality—how complete its description is, how many relationships are documented, and how many source documents support it. Confidence scoring adjusts the retrieval score: $s' = s \times (0.5 + c)$, where c is the entity’s confidence. The 0.5 floor ensures that even minimally documented entities retain half their retrieval score rather than being zeroed out; a pure multiplicative model ($s' = s \times c$) would eliminate newly created entities before curation could validate them. Additional boosts account for relationship density ($\log(1 + \text{edge_count})$) and source document count.

3.5 Stage 5: Five-Layer Deduplication

Deduplication applies five sequential filters: (1) exact entity ID dedup, (2) text similarity dedup (cosine > 0.85 on body text), (3) type diversity enforcement (limits entities per type based on intent), (4) inverse frequency weighting (deprioritizes ubiquitous entities), and (5) layer diversity enforcement (ensures representation across knowledge layers).

3.6 Stage 6: Context Building and Synthesis

Retrieved entities are assembled into a structured context window. An LLM synthesizes an answer with entity-level citations. The LLM also self-scores the answer as ANSWERABLE, PARTIAL, or NOT_ANSWERABLE, which we map to CLEAN, INCOMPLETE, and MISSING respectively.

3.7 Stage 7: Gap Detection

Gap detection examines the retrieval trace to identify three types of knowledge base deficiencies:

- **Orphan entities.** Entities retrieved with high relevance but no typed relationships to other retrieved entities—indicating missing relationship documentation.

- **Ambiguous entities.** Pairs of retrieved entities with high name or content similarity but different types—indicating potential duplicates or unclear type boundaries.
- **Thin coverage.** Retrieved entity count meets the top- K threshold but content density is low across the result set—indicating that the knowledge base lacks depth in this area.

Gap detection is downstream of retrieval and independent of answer quality. Its output is a structured list of gap signals, each with a type, severity, and description. These signals are designed for knowledge base curators, not end users.

4 Experimental Setup

4.1 Knowledge Base

We use a synthetic healthcare claims processing domain: Clearview Health Plans, a fictional regional health insurer. The knowledge base contains 408 entities across 18 types and 6 knowledge layers (Table 1).

The knowledge base was constructed in three stages. First, 148 synthetic source documents were generated describing the domain’s systems, processes, APIs, data models, teams, and terminology. Second, an automated extraction pipeline (Claude Sonnet with a typed meta-model schema) processed each document to identify entities, assign types and layers, extract relationships, and generate confidence scores based on description completeness, relationship density, and source document coverage. Third, extracted entities underwent manual curation review: deduplication, relationship validation, and quality assessment. The pipeline and source documents are included in the repository.

Table 1: Knowledge base composition by layer.

Layer	Entity Types	Entity Count
Structural	3	142
Behavioral	4	118
Reference	3	67
Organizational	3	41
Language	2	28
Decision	1	12
Total	18	408

4.2 Evaluation Questions

We generated 50 evaluation questions from seed context using an LLM, then deduplicated using a dual threshold: Jaccard similarity > 0.55 on question tokens and entity overlap > 0.7 on extracted entity references. Questions span two categories: 18 cross-cutting questions requiring synthesis across multiple entity types and layers, and 32 single-layer questions addressable from entities within a single knowledge layer.

4.3 Conditions

We evaluate 9 retrieval conditions: 3 baselines and 6 ablation variants.

Baselines.

- **Naive vector.** Cosine similarity only. No keyword search, no graph traversal, no fusion.

- **Vanilla RAG.** Vector search plus keyword search with simple score averaging. No typed traversal, no confidence scoring, no dedup beyond ID matching.
- **DCR.** Full seven-stage pipeline.

Ablation. Each variant removes one stage from the full DCR pipeline:

- **full_dcr:** control (identical to DCR baseline, run separately to measure scoring variance).
- **no_intent:** Stage 1 removed; flat layer weights.
- **no_graph:** Stage 2 graph traversal removed; vector + keyword only.
- **no_layer_boost:** Stage 3 layer priority boosting removed from fusion.
- **no_confidence:** Stage 4 confidence scoring removed.
- **no_dedup:** Stage 5 multi-layer dedup replaced with simple ID dedup.

4.4 Scoring

Each retrieved context is passed to Claude Sonnet¹ for synthesis and self-scoring in a single LLM call. The model generates an answer with entity citations and, on the final line, self-scores as ANSWERABLE, PARTIAL, or NOT_ANSWERABLE. We map these to:

- **CLEAN:** the knowledge base fully addresses the question.
- **INCOMPLETE:** the knowledge base partially addresses the question but key information is missing.
- **MISSING:** the knowledge base does not contain enough information to answer.

The same model is used for both synthesis and scoring. While this introduces a potential self-enhancement bias [Zheng et al., 2023], we mitigate it in three ways: (1) multi-trial averaging reduces single-run bias, (2) the inter-rater reliability check (Section 4.5) re-scores answers *without* retrieval context, removing the synthesis framing, and (3) we report relative comparisons across conditions rather than absolute quality claims—any systematic bias affects all conditions equally.

Each condition is run 3 independent times (different LLM scoring runs on identical retrieval results) to characterize scoring variance. Total evaluations: 50 questions $\times$ 9 conditions $\times$ 3 trials = 1,350.

4.5 Judge Consistency

We assessed inter-rater reliability by re-scoring 15 stratified (question, answer) pairs (5 CLEAN, 8 INCOMPLETE, 2 MISSING) through the same LLM judge twice. Self-consistency was 100% (15/15). Agreement with original scoring was 80% (12/15). Cohen’s $\kappa = 0.634$ (substantial agreement). We note that $n = 15$ is below the typical minimum for reliability studies; the kappa provides a directional but not statistically powered reliability estimate. All 3 disagreements were one-directional promotions of borderline cases (MISSING $\rightarrow$ INCOMPLETE or INCOMPLETE $\rightarrow$ CLEAN), indicating conservative original scoring.

Cross-model validation. To address same-model self-bias, we scored 30 (question, answer) pairs using GPT-4o as an independent judge: 15 from DCR and 15 from vanilla RAG, each stratified as 5 CLEAN, 7 INCOMPLETE, 3 MISSING. Cross-model agreement with Claude’s original scoring was 80.0% (24/30), identical to Claude’s self-agreement rate. All 6 disagreements were promotions (3 per condition), affecting both conditions equally. GPT-4o confirms the conservative scoring pattern without distorting the relative comparison between retrieval pipelines.

¹Model ID: `claude-sonnet-4-20250514` (Anthropic’s Claude Sonnet 4 release, May 2025). Accessed via Anthropic API.

5 Results

5.1 Baseline Comparison

Table 2 shows the baseline comparison. DCR achieves the lowest MISSING rate ($4.7\% \pm 2.3$) with modest CLEAN improvement over baselines. Note that LLM scoring variance introduces a ~ 4 pp noise floor: the ablation’s full_dcr condition (Table 3), an identical pipeline scored in a separate trial batch, reports 38.0% CLEAN vs. 34.0% here. Section 5.4 discusses this variance in detail.

Table 2: Baseline comparison (mean $\pm$ SD across 3 trials, $n = 50$ questions).

Condition	CLEAN (%)	INCOMPLETE (%)	MISSING (%)
Naive Vector	30.0 ± 3.5	60.7 ± 2.3	9.3 ± 3.1
Vanilla RAG	30.0 ± 0.0	57.3 ± 2.3	12.7 ± 2.3
DCR (ours)	34.0 ± 2.0	61.3 ± 4.2	4.7 ± 2.3

The MISSING reduction from vanilla RAG to DCR is statistically significant. We constructed a 2×2 contingency table (MISSING vs. non-MISSING $\times$ DCR vs. vanilla RAG, pooling all 150 observations per condition): DCR produced 7 MISSING and 143 non-MISSING; vanilla RAG produced 19 MISSING and 131 non-MISSING. Chi-square test: $\chi^2 = 5.10$, $df = 1$, $p = 0.024$.

For overall score comparison, we encoded DDC classes as ordinal values (CLEAN = 3, INCOMPLETE = 2, MISSING = 1) and applied the Mann-Whitney U test across 150 observations per condition: $U = 10307.5$, $p = 0.059$, not significant at $\alpha = 0.05$.

5.2 Ablation Study

Table 3 shows the effect of removing individual pipeline stages.

Table 3: Ablation study—stage contribution (mean $\pm$ SD across 3 trials). The full_dcr row is an identical pipeline to DCR in Table 2, scored in a separate trial batch; the 4pp CLEAN difference reflects LLM scoring variance.

Condition	CLEAN (%)	INCOMPLETE (%)	MISSING (%)	Δ CLEAN
full_dcr	38.0 ± 6.0	56.7 ± 6.4	5.3 ± 4.2	—
no_intent	44.7 ± 4.2	49.3 ± 4.2	6.0 ± 2.0	+6.7
no_graph	40.7 ± 1.2	53.3 ± 3.1	6.0 ± 2.0	+2.7
no_layer_boost	37.3 ± 3.1	59.3 ± 4.2	3.3 ± 1.2	−0.7
no_confidence	35.3 ± 4.2	56.0 ± 5.3	8.7 ± 1.2	−2.7
no_dedup	39.3 ± 1.2	56.0 ± 2.0	4.7 ± 1.2	+1.3

Two findings stand out. First, removing intent classification *increases* CLEAN by 6.7 percentage points, and removing graph traversal increases it by 2.7pp. This counterintuitive result is discussed in Section 7.1. Second, removing confidence scoring decreases CLEAN by 2.7pp *and* increases MISSING to 8.7%—the highest among all ablation conditions. Confidence scoring is the one stage whose removal degrades both precision and failure prevention.

5.3 Scoring Variance

The baseline DCR condition and the ablation full_dcr condition run identical pipeline code. Their CLEAN scores differ by 4 percentage points (34.0% vs. 38.0%), with full_dcr exhibiting

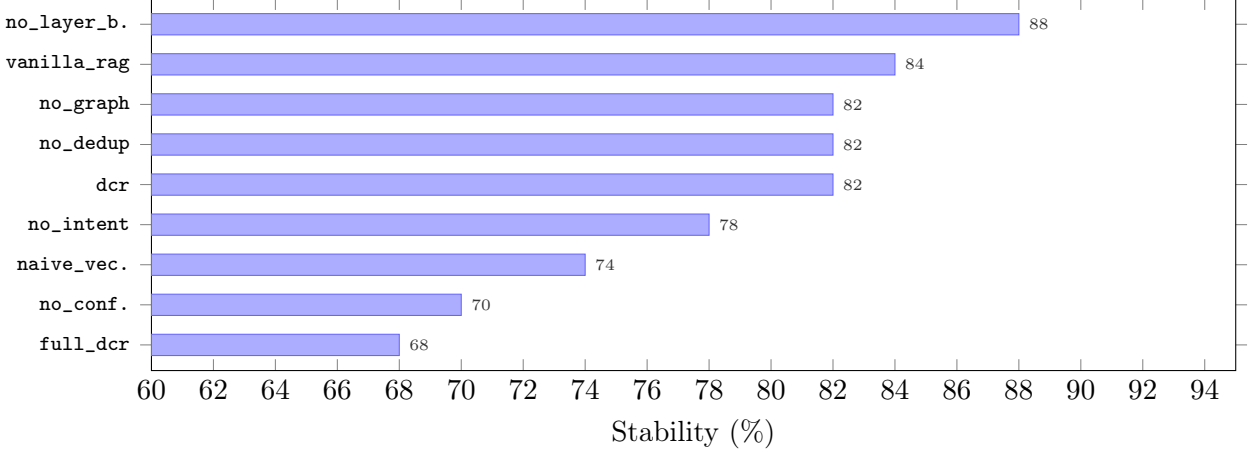


Figure 3: Per-question stability (percentage of questions with identical DDC class across all 3 trials). Simpler pipelines achieve higher consistency; the full pipeline is least stable at 68%.

higher variance ($SD = 6.0$ vs. 2.0). This gap—pure LLM scoring variance on identical retrieval—establishes the noise floor for interpreting all results.

5.4 Per-Question Stability

Table 4 reports the percentage of questions that received the same DDC class across all 3 trials.

Table 4: Per-question stability (same DDC class across 3 trials).

Condition	Stable Questions	Stability (%)
no_layer_boost	44/50	88
vanilla_rag	42/50	84
dcr	41/50	82
no_dedup	41/50	82
no_graph	41/50	82
no_intent	39/50	78
naive_vector	37/50	74
no_confidence	35/50	70
full_dcr	34/50	68

Figure 3 visualizes the stability data. Pipeline complexity inversely correlates with stability: simpler conditions (vanilla RAG, no_layer_boost) are more stable than complex ones (full DCR). This suggests that more pipeline stages introduce more variance in the context passed to the LLM judge, even when the retrieval is deterministic.

6 Analysis

6.1 Question Difficulty Stratification

We stratified questions into two categories: 18 cross-cutting questions requiring synthesis across entity types and layers, and 32 single-layer questions addressable within a single knowledge layer. Table 5 shows the stratified baseline results.

Figure 4 visualizes the stratified MISSING rates. The MISSING reduction concentrates overwhelmingly in single-layer questions: from 12.5% (vanilla RAG) to 1.0% (DCR), an 11.5 percentage-point drop.

Table 5: Stratified baseline results by question type.

Group	Condition	CLEAN (%)	INC (%)	MISS (%)
Cross-cutting ($n = 18$)	Naive Vector	3.7	83.3	13.0
	Vanilla RAG	1.9	85.2	13.0
	DCR	5.6	83.3	11.1
Single-layer ($n = 32$)	Naive Vector	44.8	47.9	7.3
	Vanilla RAG	45.8	41.7	12.5
	DCR	50.0	49.0	1.0

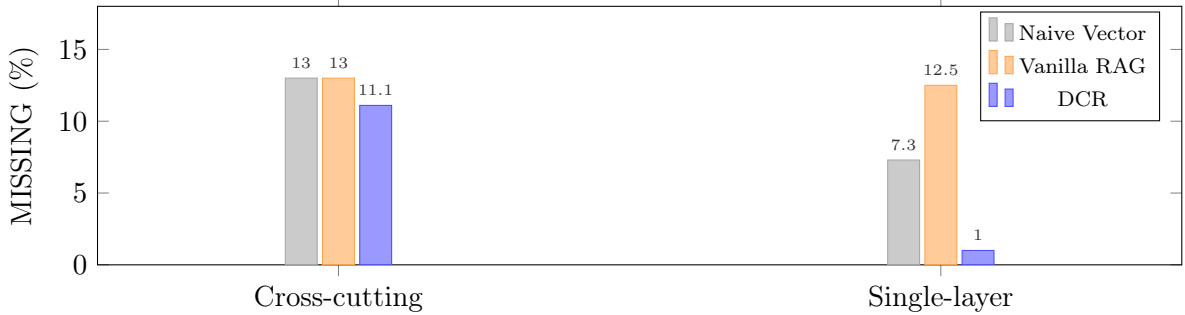


Figure 4: MISSING rates stratified by question type. DCR’s failure reduction concentrates in single-layer questions (12.5% $\rightarrow$ 1.0%), while cross-cutting questions remain at $\sim$ 13% MISSING regardless of method—a knowledge base content ceiling, not a retrieval limitation.

Cross-cutting questions show only a 1.9pp reduction (13.0% $\rightarrow$ 11.1%). This separates two distinct phenomena:

- **Retrieval failure** (which DCR fixes): on single-layer questions where the answer exists in the knowledge base, DCR nearly eliminates the case where the pipeline fails to retrieve it.
- **Knowledge base incompleteness** (which DCR diagnoses but cannot fix): cross-cutting questions require integrated process documentation that the knowledge base lacks. The $\sim$ 83% INCOMPLETE rate is a content ceiling, not a retrieval limitation.

6.2 Confidence Scoring Across Strata

Stratified ablation on cross-cutting questions reveals that removing confidence scoring doubles MISSING from 7.4% to 16.7%—the largest single-stage effect in the entire experiment. Confidence scoring is content-density-aware rather than type-aware: it adjusts entity relevance based on how well-documented the entity is, not what type it belongs to. This makes it effective regardless of knowledge base size, and particularly valuable on hard questions where retrieved entities tend to be thin.

6.3 Gap Detection

Gap detection fires on 82% of questions (41 of 50), producing 184 gap signals across 3 trials. Table 6 shows the gap type distribution. Figure 5 illustrates the three gap types with concrete examples from the experiment.

The 9 questions (18%) where gap detection does not fire are exclusively CLEAN or INCOMPLETE—no MISSING question escapes detection. These are single-entity definitional questions where the retrieved entities have well-documented relationships and sufficient depth.

Table 6: Gap detection results across 3 DCR trials.

Gap Type	Count	Severity	Description
Orphan entity	90	low	No typed relationships
Ambiguous entity	87	medium	Name/content overlap across types
Thin coverage	7	high	Content density insufficient
Total	184		

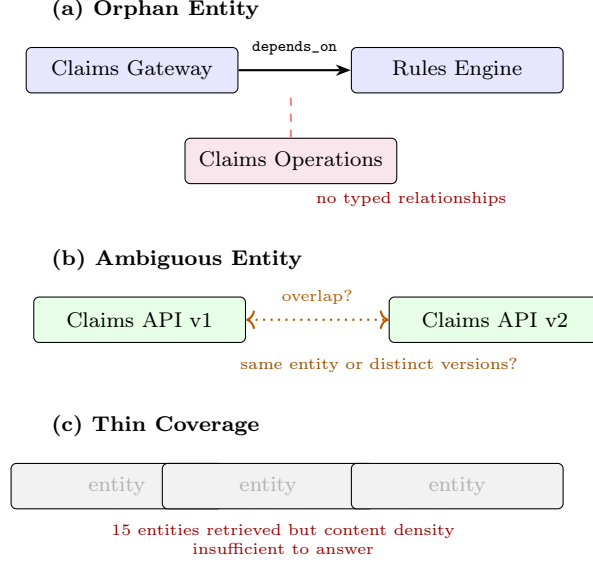


Figure 5: The three gap detection types with examples from the experiment. (a) Orphan entities have high retrieval relevance but no typed relationships to other results. (b) Ambiguous entities share names or content across different types. (c) Thin coverage: the entity count meets the threshold but content lacks depth. Thin coverage gaps (c) correlate with MISSING outcomes (7/7).

A counterintuitive finding: gap count does *not* correlate with MISSING scores. CLEAN answers average 1.39 gaps per question, while MISSING answers average 1.29. This means gap detection is not measuring retrieval failure—it is measuring knowledge base structural health. Orphan entities and ambiguous types surface regardless of whether the current query succeeds.

However, **thin coverage gaps do predict retrieval failure**: all 7 thin-coverage events in the dataset co-occurred with MISSING outcomes. While $n = 7$ is too small for statistical claims, this perfect correlation suggests high-severity coverage gaps as a potential predictor of retrieval failure. We note this should be validated on larger question sets.

Gap detection is downstream of intent classification: the `no_intent` ablation condition detects 126 gaps compared to 184–185 for conditions with intent enabled. Without intent classification, the pipeline lacks the type expectations needed to identify orphan entities or ambiguous type assignments. This is expected behavior—a design dependency, not a deficiency.

6.4 INCOMPLETE Characterization

INCOMPLETE answers constitute 50–65% of results across all conditions—the largest category. To characterize this mass, we qualitatively examined 10 randomly sampled INCOMPLETE answers from DCR. This characterization is based on qualitative inspection and should not be interpreted as a quantitative finding; no second reviewer independently assessed these cases.

All 10 share the same pattern: correct entity retrieval (15 hops per question), reasonable

synthesis with entity-level citations, and explicit self-identification of what specific detail is missing. Examples include: “800K members across three states” with the gap being that the specific state names are not documented; a full claims processing flow missing only the final payment issuance steps; a correct definition of Coordination of Benefits missing operational exception handling.

No INCOMPLETE answer was near-MISSING. The INCOMPLETE plateau represents a **knowledge base content ceiling**: the pipeline retrieves the right entities, but those entities lack operational depth. This is precisely the signal that gap detection is designed to surface, and it reinforces DCR’s role as a curation tool: it tells you not just what it found, but what you should document next.

7 Discussion

7.1 Density Dependence of Typed Retrieval Stages

The ablation study reveals a counterintuitive result: removing intent classification and graph traversal *improves* CLEAN scores on this 408-entity knowledge base. This does not mean typed retrieval is harmful. It means typed stages are density-dependent.

On a dense knowledge base where entities of every type are well-represented, intent-based filtering can push relevant entities down the ranking by suppressing types that fall outside the predicted intent. For example, a process query might deprioritize *system* entities that are, in fact, essential to answering the question. Graph traversal compounds this by expanding the result set along typed edges that may lead away from the most relevant entities.

We hypothesize that typed retrieval stages earn their contribution on bounded-context knowledge bases containing 50–80 entities, where disambiguation between similar entities matters and the graph is sparse enough for typed traversal to add signal rather than noise. Testing this hypothesis requires experiments on partitioned knowledge bases and is left to future work.

This distinction—between the value of a *typed ontology* (entity types, layers, relationships, curated descriptions) and the value of *typed retrieval stages* (intent classification, layer boosting, graph traversal)—is important. The typed ontology makes *every* retrieval method better: even naive vector search achieves 30% CLEAN on this well-structured knowledge base. The typed retrieval stages are an additional layer of pipeline complexity whose benefit depends on knowledge base density.

7.2 Gap Detection as Curation Signal

Gap detection’s 82% trigger rate and its independence from answer quality position it as a continuous knowledge base health probe rather than a retrieval confidence signal. Every query surfaces structural information about the knowledge base: orphan entities needing relationship documentation, ambiguous types needing clarification, and thin coverage areas needing content investment.

The thin-coverage correlation with MISSING (7/7) suggests a hierarchy of gap severity: low-severity gaps (orphans, ambiguities) are pervasive and represent ongoing curation work, while high-severity gaps (thin coverage) indicate areas where the knowledge base cannot support reliable retrieval. This hierarchy has practical implications for knowledge base maintenance: curators can prioritize thin-coverage gaps for immediate attention while treating orphan and ambiguity gaps as background improvement work.

7.3 Evaluation Methodology

Our multi-trial design with per-question stability analysis reveals that pipeline complexity inversely correlates with scoring consistency. Full DCR achieves 68% stability compared to 84%

for vanilla RAG and 88% for the simplest ablation condition. This suggests that more complex retrieval produces more varied context, which in turn produces more varied LLM scoring.

This observation has implications for RAG evaluation practice: single-trial evaluations of complex pipelines may not be reliable, and evaluation frameworks should report stability alongside accuracy. The 4-percentage-point gap between baseline DCR and ablation full_dcr—identical pipelines scored in different trial batches—illustrates the magnitude of this variance.

7.4 Limitations

- **Single knowledge base.** All experiments use one synthetic domain. Generalization to other domains, knowledge base sizes, and entity type distributions is not established.
- **Synthetic data.** The knowledge base was generated and curated, not extracted from a production enterprise. Production knowledge bases may have different quality distributions.
- **LLM judge with self-scoring.** The same model (Claude Sonnet) generates and scores answers, introducing potential self-enhancement bias [Zheng et al., 2023]. Our cross-model check (GPT-4o, $n = 30$) shows 80% agreement with identical promotion patterns across conditions, suggesting the bias does not distort relative comparisons. However, absolute DDC class rates may differ under human evaluation.
- **No K sensitivity analysis.** The top- K retrieval threshold is auto-scaled to 20% of entity count. This default was not ablated; different K values may affect both baseline and DCR results.
- **Small question set.** 50 questions provide adequate statistical power for the MISSING comparison but limit analysis of per-question-type effects. The thin-coverage finding ($n = 7$) requires validation at scale.
- **LLM-generated questions.** The 50 evaluation questions were generated by an LLM from seed context rather than drawn from real user queries. This may bias the question type distribution toward patterns the model finds syntactically natural, underrepresenting the kinds of ad hoc or poorly phrased questions that arise in production enterprise settings.
- **Density hypothesis untested.** The claim that typed stages help on smaller knowledge bases is a hypothesis motivated by the ablation results, not an empirical finding in this paper.

8 Conclusion

We presented Domain-Constrained Retrieval, a seven-stage pipeline for typed, layered knowledge bases. DCR significantly reduces catastrophic retrieval failures ($p = 0.024$) and converts every query into a knowledge base health probe via gap detection. The MISSING reduction concentrates in single-layer questions ($12.5\% \rightarrow 1.0\%$), while cross-cutting incompleteness exposes genuine knowledge base content gaps that no retrieval method can fix.

Stage-by-stage ablation reveals that contributions are density-dependent: confidence scoring is the load-bearing stage, while intent classification and graph traversal add noise on dense knowledge bases. This characterizes a retrieval design space rather than a pipeline failure—typed stages are hypothesized to help on bounded-context knowledge bases where disambiguation matters.

Gap detection, DCR’s novel contribution, fires on 82% of questions regardless of answer quality. It is a curation signal, not a retrieval confidence metric: it tells knowledge base curators what to document next, not whether the current answer is reliable. The thin-coverage gap type predicts retrieval failure with a perfect (7/7) but small-sample correlation.

Future work includes validating the density hypothesis by partitioning the knowledge base into bounded contexts of 50, 100, and 200 entities and re-running the same 50-question evaluation

at each scale—if typed stages improve CLEAN on smaller partitions, it confirms that pipeline contributions are density-dependent rather than architecture-dependent. We also plan to scale the evaluation to 200+ questions with deterministic scoring rubrics and human-calibrated labels, and to validate gap detection’s predictive power (particularly the thin-coverage $\rightarrow$ MISSING correlation) on larger datasets. We release the pipeline implementation, evaluation data, and synthetic knowledge base to support reproducibility.²

References

- A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In *ICLR*, 2024.
- A. Bordes, N. Usunier, A. Garcia-Duran, J. Weston, and O. Yakhnenko. Translating embeddings for modeling multi-relational data. In *NeurIPS*, 2013.
- J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu. BGE M3-Embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation. *arXiv preprint arXiv:2402.03216*, 2024.
- G. V. Cormack, C. L. A. Clarke, and S. Büttcher. Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. In *SIGIR*, 2009.
- D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, and J. Larson. From local to global: A graph RAG approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024.
- Z. Hu, Z. Xu, G. Qi, and K. Wang. GRAG: Graph retrieval-augmented generation. *arXiv preprint arXiv:2405.16506*, 2024.
- R. Navakoti and S. Navakoti. Demand-driven context: A methodology for building enterprise knowledge bases through agent failure. *arXiv preprint arXiv:2603.14057*, 2026.
- B. Sarmah, B. Hall, R. Rao, S. Patel, S. Pasupuleti, and M. Kulkarni. HybridRAG: Integrating knowledge graphs and vector retrieval augmented generation for efficient information extraction. *arXiv preprint arXiv:2408.04948*, 2024.
- P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *NeurIPS*, 2020.
- S. Liu, X. Li, Z. Liu, Y. Yan, C. Yang, Z. Zeng, Z. Liu, M. Sun, and G. Yu. Judge as a judge: Improving the evaluation of retrieval-augmented generation through the judge-consistency of large language models. In *Findings of ACL*, pages 5788–5807, 2025.
- O. Press, M. Zhang, S. Min, L. Schmidt, N. A. Smith, and M. Lewis. Measuring and narrowing the compositionality gap in language models. In *Findings of EMNLP*, 2023.
- N. Rajkumar, R. Li, and D. Baber. Evaluating the text-to-SQL capabilities of large language models. *arXiv preprint arXiv:2204.00498*, 2022.
- P. Rajpurkar, R. Jia, and P. Liang. Know what you don’t know: Unanswerable questions for SQuAD. In *ACL*, 2018.

²<https://github.com/ea-toolkit/dcr-paper>

- S. Shankar, J. Zamfirescu-Pereira, B. Hartmann, A. Parameswaran, and I. Arawjo. Who validates the validators? Aligning LLM-assisted evaluation of LLM outputs with human preferences. In *UIST*, 2024.
- L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *NeurIPS*, 2023.